Table 1 Users

UID (PK)	Username (Unique Not Null)	Password Not Null	Gmail unique not null.

UIDS, strings instead of ints, more characters, more options.
Green: pass in from front end to back end to database
Allows us to return golden key if need

Table 2 Private Pins

PID	UID (Creator)	Category	long	lat	timestamp	Radius

SECONDARY GOAL: PIDS, strings instead of ints, more characters, more options. Random generated so time isn't a factor.

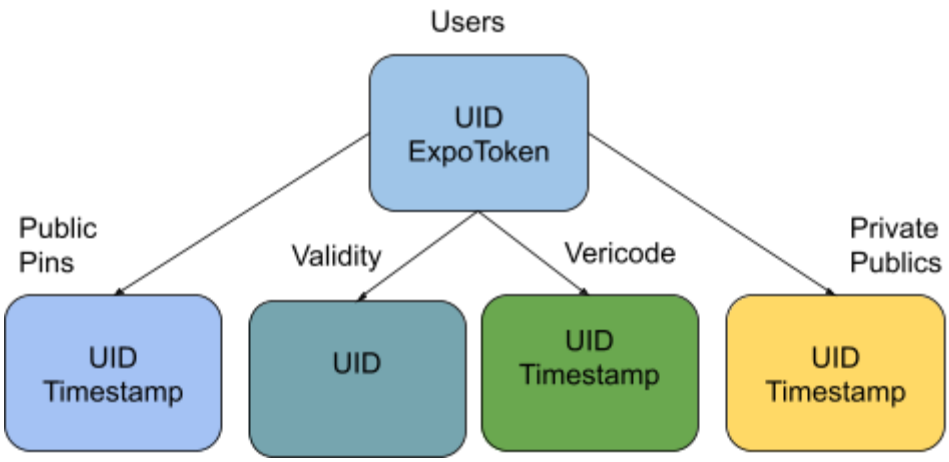
Table 3 Public Pins

PID	UID	Category	long	lat	timestamp	likes

Likes (Can be positive or negative, like reddit. If < -5. Remove on cleanup when fetched)

Table 4 Validity

UID	Endorsed Pins (SETS)
61	P1-, P2-, P3




```
CREATE OR REPLACE FUNCTION clean_and_fetch_public_pins()
RETURNS SETOF PublicPins AS
$$
BEGIN
    DELETE FROM PublicPins
    WHERE timestamp < NOW() - INTERVAL '3 hours';

    RETURN QUERY
    SELECT * FROM PublicPins;
END;
$$ LANGUAGE pgsql;
```

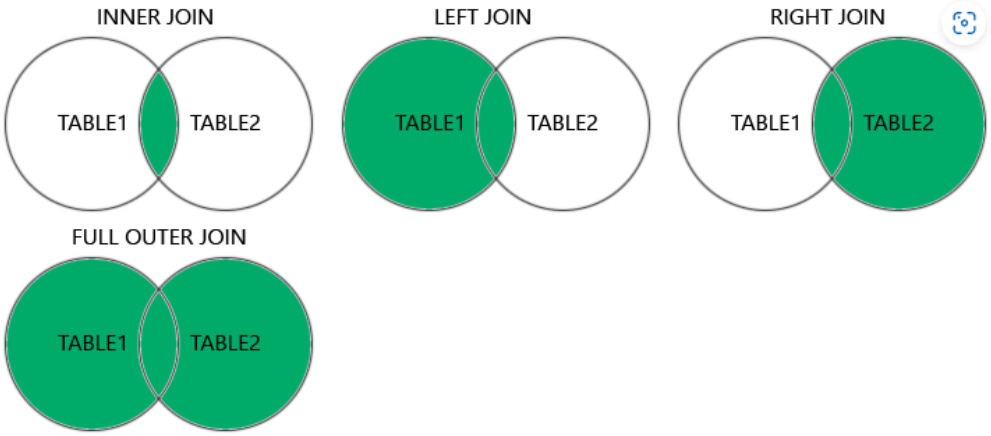
Validity

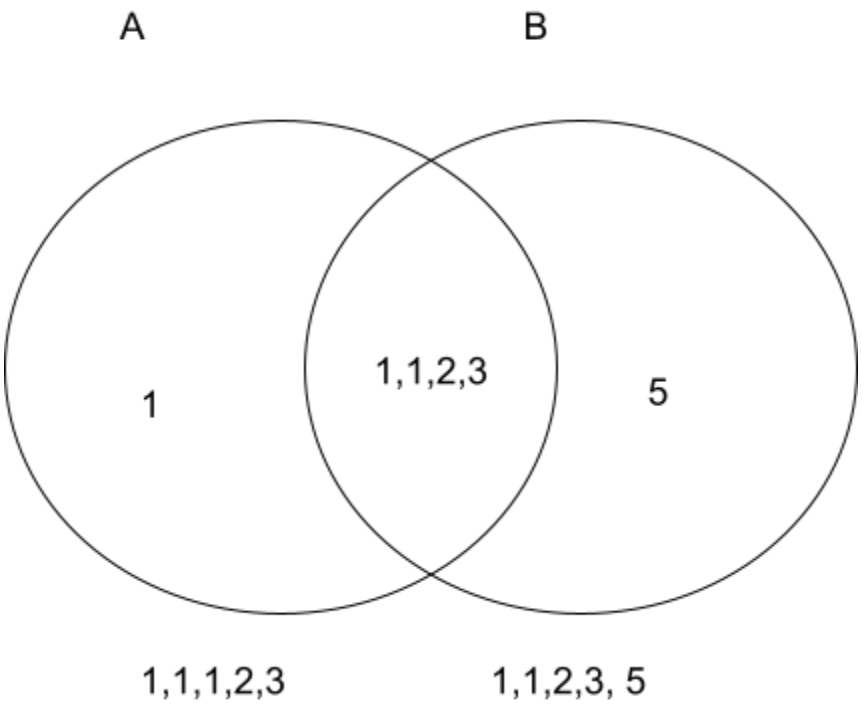
When needing information from two separate tables with one query, you can use:

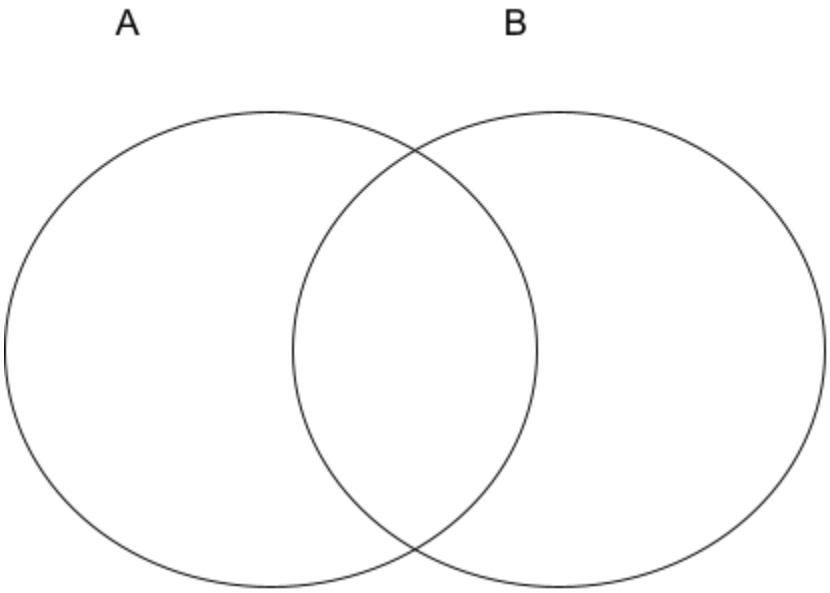
Different Types of SQL JOINs

Here are the different types of the JOINs in SQL:

- `(INNER) JOIN`: Returns records that have matching values in both tables
- `LEFT (OUTER) JOIN`: Returns all records from the left table, and the matched records from the right table
- `RIGHT (OUTER) JOIN`: Returns all records from the right table, and the matched records from the left table
- `FULL (OUTER) JOIN`: Returns all records when there is a match in either left or right table







C	A	B	C
1	1	2	3
2	2	3	4
3	4	5	6

C	C	D	E
1	1	2	24
2	3	3	9
3	6	5	6

C	A	B	C	||	D	E
1	1	2	3		3	9
2	2	3	4		/	/
3	4	5	6		5	6

View: Glass Table, stays up, changes on updates, but does not take as much data. Cannot insert data into views.

Adding points to validity,
Return validity, long, lat.

pgSQL without proper data structure wont let you touch Primary Key for a table to insert your own values, check fixes in database schema.

DATABASE SCHEMA:

```
-- USERS TABLE
CREATE TABLE users (
    uid VARCHAR(100) PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    password VARCHAR(100) NOT NULL,
    gmail VARCHAR(100) NOT NULL,
    expoToken VARCHAR(50)
);

-- PRIVATE PINS TABLE
CREATE TABLE private_pins (
    pid VARCHAR(100) PRIMARY KEY,
    uid VARCHAR(100) REFERENCES users(uid) ON DELETE CASCADE,
    category VARCHAR(50) NOT NULL,
    longitude DOUBLE PRECISION NOT NULL,
    latitude DOUBLE PRECISION NOT NULL,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    radius DOUBLE PRECISION DEFAULT 0
);

-- PUBLIC PINS TABLE
CREATE TABLE public_pins (
    pid VARCHAR(100) PRIMARY KEY,
    uid VARCHAR(100) REFERENCES users(uid) ON DELETE CASCADE,
    category VARCHAR(50) NOT NULL,
    longitude DOUBLE PRECISION NOT NULL,
    latitude DOUBLE PRECISION NOT NULL,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    likes INT DEFAULT 0
);

-- VALIDITY TABLE
CREATE TABLE validity (
    uid VARCHAR(100) REFERENCES users(uid) ON DELETE CASCADE,
    endorsed_pins TEXT[] DEFAULT '{}',
    PRIMARY KEY (uid)
);

-- VERICODE TABLE
CREATE TABLE varicode (
    code varchar(255),
    uid VARCHAR(100) REFERENCES users(uid) ON DELETE CASCADE,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

);
//THIS DOES NOT REMOVE INTERACTED PINS.

For the endorsed pins, its an array, I dont know if this will work but from documentation it seem slike this is ok. By making it varchars for PIDs and UIDs, we need to make our own.
pgSQL should handle the array size and so we never need to do anything like that, just updates. And it said if we're doing a million size array then its not worth it but we arnt.

Inserts for arrays hopefully:
Insert new validity
INSERT INTO validity (uid) VALUES ('user_abc');

Add pin_123 to endorsed pins
UPDATE validity SET endorsed_pins = endorsed_pins || 'pin_123'
WHERE uid = 'user_abc';

Remove pin_123
UPDATE validity SET endorsed_pins = array_remove(endorsed_pins, 'pin_123') WHERE uid = 'user_abc';

Find users who endorsed pin_456 //find users
SELECT * FROM validity WHERE 'pin_456' = ANY(endorsed_pins);

H Insert into users
H Fetch from users
H Fetch from public pins
N Fetch from private pins

SQL Transaction

SET TRANSACTION READ ONLY / READ WRITE
ISOLATION LEVEL <Isolation_Level>;
BEGIN TRANSACTION;
<SQL statements>
COMMIT / ROLLBACK;

```
SET TRANSACTION READ ONLY
    ISOLATION LEVEL REPEATABLE READ;
BEGIN TRANSACTION;

SELECT * FROM Accounts WHERE balance > 500;

COMMIT;
```

SQL Isolation Levels

	Dirty Reads	Nonrepeatable Reads	Phantom Reads
Serializable	Not Allowed	Not Allowed	Not Allowed
Repeatable Read	Not Allowed	Not Allowed	Allowed
Read Committed	Not Allowed	Allowed	Allowed
Read Uncommitted	Allowed	Allowed	Allowed



More Parallelization

- SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

- SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
  - Repeated queries of a tuple during a transaction will retrieve the same value, even if its value was changed by another transaction.
  - But reads of *different* data might return values that were committed by *different* transactions at *different* times.
  - A second scan of a range (e.g., salary > 10000) might return "phantoms" not originally present in the first scan.

SQL Isolation Levels

	Dirty Reads	Nonrepeatable Reads	Phantom Reads
Serializable	Not Allowed	Not Allowed	Not Allowed
Repeatable Read	Not Allowed	Not Allowed	Allowed
Read Committed	Not Allowed	Allowed	Allowed
Read Uncommitted	Allowed	Allowed	Allowed



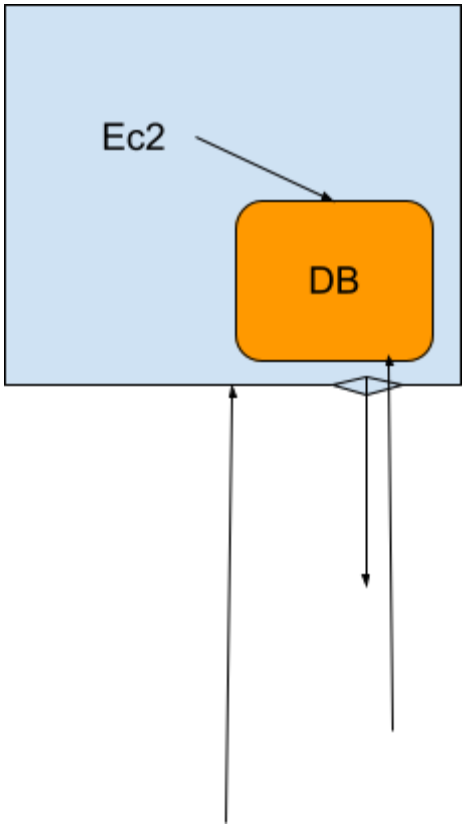
More Parallelization

- SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
  - Only clean (committed) reads, no dirty reads.
  - But you might read data committed by *different* transactions.
  - You might not get the same value even when you read the same data second time during a single transaction!

- SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
  - May read *dirty data*
  - Read data may be inconsistent if rolled back by the other transaction
  - Acceptable where dirty data is not catastrophic but high parallelism is preferred

Comparison Operators

- x IN Q
  - Returns true if x occurs in the collection Q.
- x NOT IN Q
  - Returns true if x does not occur in the collection Q.
- x > ANY Q
  - Returns TRUE if x is greater than at least one value in Q.
  - Can also use >=, <, <=, =, <>
- x > ALL Q
  - Returns TRUE if x is greater than every value in Q.
  - Can also use >=, <, <=, =, <>
- EXISTS Q
  - Returns true if Q is a non-empty collection.
- NOT EXISTS Q
  - Returns true if Q is an empty collection.



Front end, mark public or private.

WATCH AREA: A certain radius within the pin.
Radius